DevOps Final Capstone: TaskBoard

- A SaaS-style project management application built with Python and modern DevOps practices.
  - Frontend: React + Vite with responsive dashboard UI
  - Backend: FastAPI with REST APIs and Swagger docs
  - Database: PostgreSQL with SQLAlchemy and Alembic migrations
  - Testing: Pytest automated tests
  - Deployment: Docker, GitHub Actions, Terraform, AWS EKS
  - Operations: Helm, Ingress, HPA, Prometheus, and Grafana
- Goal: Show how an application travels from a developer laptop to a monitored Kubernetes environment.

Application Overview

- TaskBoard provides a realistic project-management dashboard.
- Frontend features:
  - Dark sidebar, workspace navigation, and dashboard header
  - KPI cards, task table, status filters, and priority badges
  - Activity feed, pipeline indicator, and create-task modal
  - Loading and backend-error states
- Backend APIs:
  - GET /api/tasks and GET /api/tasks/{id}
  - POST /api/tasks, PUT /api/tasks/{id}, DELETE /api/tasks/{id}
  - GET /api/tasks/stats for dashboard statistics
- Health endpoints: /health for liveness, /ready for readiness, /metrics for Prometheus.

Local Development and Testing

- Docker Compose is the fastest way to run the complete stack locally.
  - Run: docker compose up --build
  - Frontend: http://localhost:3000
  - Backend Swagger: http://localhost:8000/docs
  - Backend health: http://localhost:8000/health
  - Backend metrics: http://localhost:8000/metrics
- Stop stack: docker compose down; remove database volume with docker compose down -v.
- Backend can also run directly using Python 3.12+, PostgreSQL, Alembic, and Uvicorn.
- Pytest runs before image creation and acts as the first quality gate.
  - If tests fail, the CI pipeline stops and no broken image is promoted.

Git, Docker, and CI/CD

- Git provides version control; GitHub provides remote collaboration and automation.
  - Key concepts: commit, branch, pull request, and controlled change review.
- Docker packages the application into portable container images.
  - Backend image: Python, dependencies, Alembic, application code, non-root user, Uvicorn.
  - Frontend image: multi-stage build from Node build output to a lightweight Nginx runtime.
- GitHub Actions pipeline stages:
  - TEST: run Pytest and build the React frontend
  - BUILD + SECURITY SCAN + PUSH: build images, scan with Trivy, push to GHCR
  - DEPLOY: install Helm, configure kubectl, and run helm upgrade --install
- Images are tagged with the Git commit SHA for traceability from production back to source code.

Security, Terraform, and AWS

- Trivy scans container images for HIGH and CRITICAL vulnerabilities.
  - Security has multiple layers: SAST, dependency scanning, secret scanning, container scanning, and runtime security.
- Terraform is Infrastructure as Code: infrastructure is described in code and reconciled to the desired state.
- Terraform creates:
  - AWS VPC with public and private subnets
  - NAT gateway
  - EKS cluster with managed worker nodes
  - Workflow: terraform init, terraform plan, terraform apply.
  - Default AWS region: ap-south-1.
- After EKS is created, configure kubectl using the AWS/Terraform output command.
- Use terraform destroy to remove infrastructure when finished.

Kubernetes, Helm, Ingress, and HPA

- A namespace provides logical isolation for the TaskBoard application.
- Helm packages Kubernetes resources into a reusable chart with values, templates, releases, upgrades, and rollbacks.
  - Deploy with: helm upgrade --install taskboard ./helm/taskboard --namespace taskboard --create-namespace
- Deployments manage frontend and backend Pods and replace failed Pods automatically.
- Services provide stable networking because Pods are ephemeral.
- PostgreSQL runs in-cluster with a PVC for the classroom demo; Amazon RDS is a production alternative.
- Ingress routes:
  - /taskboard.local/ to the React frontend
  - /taskboard.local/api to the FastAPI backend
  - An Ingress resource needs an Ingress Controller to actually implement routing.
- HPA scales backend Pods based on CPU utilization and requires resource requests plus Metrics Server.

Monitoring, Troubleshooting, and Final Demo

- Prometheus collects metrics from the FastAPI /metrics endpoint using a ServiceMonitor.
- Grafana visualizes request volume, latency, errors, CPU usage, and HPA scaling.
- Troubleshooting labs:
  - Broken image: investigate ImagePullBackOff using kubectl describe pod and kubectl get events
  - Broken Service: check Services, Endpoints, and Pod labels; no matching labels means no endpoints and no traffic
- Final demonstration flow:
  - Application and API demo, database verification, Git commit, and CI run
  - Docker images, Trivy scan, GHCR registry, Terraform infrastructure, and Kubernetes deployment
  - Helm release, Ingress access, HPA, Prometheus/Grafana, and live failure simulation
- Outcome: a complete, observable path from code commit to monitored Kubernetes deployment.